Security Testing

UNIT TESTING: DEFINITION, EXAMPLES, AND CRITICAL BEST PRACTICES

A unit test is a type of software test that focuses on components of a software product. The purpose is to ensure that each unit of software code works as expected. A unit can be a function, method, module, object, or other entity in an application's source code.

Nickolay Bakharev

Published Date
July 26, 2023
Estimated Read Time
22 minutes

What Is Unit Testing?

A unit test is a type of software test that focuses on testing individual components of a software product. Software developers and sometimes QA staff write unit tests during the development process.

The 'units' in a unit test can be functions, procedures, methods, objects, or other entities in an application's source code. Each development team decides what unit is most suitable for understanding and testing their system. For example, object-oriented design tends to treat a class as the unit. A unit test usually relies on mock objects to simulate other parts of code, or integrated systems, to ensure tests remain simple and predictable.

The main goal of unit testing is to ensure that each unit of the software performs as intended and meets requirements. Unit tests help make sure that software is working as expected before it is released.

The main steps for carrying out unit tests are:

1. Planning and setting up the environment
2. Writing the test cases and scripts
3. Executing test cases using a testing framework
4. Analyzing the results

Some advantages of unit testing include:

- Early detection of problems in the development cycle
- Reduced cost
- Test-driven development
- More frequent releases
- Enables easier code refactoring
- Detects changes which may break a design contract
- Reduced uncertainty
- Documentation of system behavior

Learn more in our detailed guide to unit testing vs integration testing.

This is part of an extensive series of guides about Software Development.

In this article:

- How Unit Tests Work
- Why Unit Testing Is Important
- How Does Unit Testing Compare to Other Types of Testing?
  - Unit Testing vs Integration Testing
  - Unit Testing vs Functional Testing
  - Unit Testing vs Regression Testing
- Can You Use Unit Testing for Security?
- Unit Testing Techniques
  - Unit Testing Examples
  - Android Unit Testing
  - Angular Unit Testing
  - Node JS Unit Testing
  - React Native Unit Testing
- Unit Testing Best Practices
  - Write Readable Tests
  - Write Deterministic Tests
  - Unit Tests Should Be Automated
- Don't Use Multiple Asserts in a Single Unit Test
- Security Unit Testing with Bright Security

How Unit Tests Work

Unit tests usually consist of four phases:

1. **Planning and setting up the environment**—developers consider which units in the code they need to test, and how to execute all relevant functionality of each unit to test it effectively.
2. **Writing the test cases and scripts**—developers write the unit test code and prepare the scripts to execute the code.
3. **Executing the unit testing**—the unit test runs and reveals how the code behaves for each test case.
4. **Analyzing the results**—developers can identify errors or issues in the code and fix them.

Test-driven development (TDD) is a common approach to unit testing. It requires the developer to create the unit test before the application code actually exists. Naturally, that initial test will fail. Then the developer adds the relevant functionality to the application until the tests pass. TDD usually results in a high quality, consistent codebase.

Effective unit testing typically:

- **Runs each test case in an isolated manner**, with "stubs" or "mocks" used to simulate external dependencies. This ensures the unit tests only considers the functionality of the current unit under test.
- **Does not test every line of code,** focusing on critical features of the unit under test. In general, unit testing should focus on code that affects the behavior of the overall software product.
- **Verifies each test case** using criteria determined in code, known as "assertions". The testing framework uses these to run the test and report failed tests.
- **Runs frequently** and early in the development lifecycle.

When a software project has been thoroughly unit tested, developers know that each individual unit is efficient and error-free. The next step is to run integration tests that evaluate larger components of the program and how they interact.

Benefits of Unit Testing

Advantages of unit testing include:

- **Detecting problems early in the development cycle**—unit testing helps in identifying bugs and issues at an early stage of the software development cycle. This early detection is crucial as it allows for issues to be addressed before they escalate into more complex problems in later stages of development.
- **Reducing costs**—by catching bugs early, unit testing can significantly reduce the cost of bug fixes. It is generally more expensive to fix bugs in later stages of development or after the software has been deployed.
- **Promoting test-driven development**—unit testing is a core component of TDD, where tests are written before the actual code. This approach ensures that the codebase is designed to pass the tests, leading to better structured, more reliable, and easier to maintain code.
- **Enabling more frequent releases**—with a comprehensive suite of unit tests, developers can make changes to the code with more confidence. This reduces the risks associated with new releases, thereby allowing for more frequent updates and improvements to the software.
- **Enabling code refactoring**—unit tests provide a safety net that allows developers to refactor code with confidence, knowing that changes can be quickly tested to ensure they don't break existing functionality encourages improving and optimizing the code without fear of introducing bugs.
- **Detecting changes that break a design contract**—unit tests can help in identifying changes in the code that may violate the intended design or contract of a system. This ensures that individual components of the software work as expected and in harmony with each other.
- **Reducing uncertainty**—with a robust unit testing process, developers gain confidence in the quality and functionality of their code. This reduces uncertainty and guesswork, especially when making changes or adding new features.
- **Documenting system behavior**—unit tests can serve as a form of documentation for the system. By reading the tests, other developers can understand what a particular piece of code is supposed to do, which is especially useful for onboarding new team members or for reference in future development.

How Does Unit Testing Compare to Other Types of Testing?

Unit Testing vs. Integration Testing

Integration testing involves testing software modules and the interaction between them. It tests groups of logically integrated modules.

Integration tests are also called thread testing, because they focus on communication between software components. Integration testing is important because most software projects consist of several independent, connected modules.

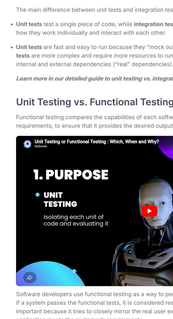
The main difference between unit tests and integration tests is what and how they test:

- **Unit tests** test a single piece of code, while **integration tests** test modules of code to understand how they work individually and interact with each other.
- **Unit tests** are fast and easy to run because they "mock out" external dependencies. **Integration tests** are more complex and require more resources to run because they must consider both internal and external dependencies ("real" dependencies).

Learn more in our detailed guide to unit testing vs. integration testing (coming soon)

Unit Testing vs. Functional Testing

Functional testing compares the capabilities of each software to the original specifications or user requirements, to ensure that it provides the desired output to end users.



Software developers use functional testing as a way to perform quality assurance (QA). Typically, if a system passes the functional tests, it is considered ready to release. Functional testing is important because it tries to closely mirror the real user experience, so it verifies that the application meets the customer's requirements.

The difference between unit testing and functional testing can be summarized as follows:

- **Unit tests** are designed to test single units of code in isolation. They are quick and easy to create, and help find and fix bugs early in the development cycle. They are typically run together with every software build. However, they are not a substitute for functional testing because they do not test the application end-to-end.
- **Functional testing** aims to test the functionality of an entire application. It is time consuming to create and requires significant computing resources to run, but is highly useful for testing the entire application flow. Functional testing is an essential part of an automated test suite, but is typically used later in the development lifecycle, and run less frequently than unit tests.

Learn more in our detailed guide to unit testing vs. functional testing (coming soon)

Unit Testing vs Regression Testing

Regression testing is a type of software testing that evaluates whether a change in the application introduced defects. It is used to determine if code changes can harm or interfere with the way an application behaves or consumes resources. In general, unit tests are regression tests, but not all regression tests are unit tests.

Unit tests are used by developers to verify the functionality of various components in their code. This ensures that all variables, functions, and objects work as expected.

Regression tests are primarily used after a programmer has completed a certain feature. Regression testing serves as a system-wide check to ensure that components that were not affected by a recent change continue to work as expected. It can include several types of tests. As part of a regression test suite, developers can run unit tests, to verify that individual features and variables behave as expected even after the change.

Learn more in our detailed guide to unit testing vs. regression testing (coming soon)

Can You Use Unit Testing for Security?

It is common to create unit tests during development. However, these tests typically only test functionality and not other aspects of the code, such as security. Many organizations are adopting a "shift left" approach in which important aspects of a software project must be tested as early as possible in the software development lifecycle, when it is easy to remediate them.

Writing security unit tests is a great way to shift left security, ensuring that developers catch security flaws in their software before a component even enters a testing environment – not to mention a production environment.

Security unit tests take the smallest testable unit of software in an application, and determine whether its security controls are effective. Developers should build security unit tests based on known security best practices for the programming language and framework, and the security controls identified during threat modeling.

Another best practice is to perform peer reviews between developers and application security specialists. Attempt a peer review of selected test strategies and individual security tests helps detect edge cases and logical flaws that individual testers might miss. Peer reviews of testers are also a great opportunity for developers, testers, and security experts to learn from each other and expand their knowledge on latest threats and new development techniques.

Unit Testing Techniques

Structural Unit Testing

Structural testing is a white box testing technique in which a developer designs test cases based on the internal structure of the code, in a white box approach. The approach requires identifying all possible paths through the code. The tester selects test case inputs, executes them, and determines the appropriate output.

Primary structural testing techniques include:

- Statement, branch, and path testing—each statement, branch, or path in a program is executed by a test at least once. Statement testing is the most granular option.
- Conditional testing—allows a developer to selectively determine the path executed by a test, by executing code based on value comparisons.
- Expression testing—tests the application against different values of a regular expression.

Related content: Read our guide to unit testing javascript.

Functional Unit Testing

Functional unit testing is a black box testing technique for testing the functionality of an application component.

Main functional techniques include:

- Input domain testing—tests the size and type of input objects and compares objects to equivalence classes.